INHERITANCE, ITERATORS, & GENERATORS

COMPUTER SCIENCE MENTORS 61A

October 19, 2026 – October 23, 2026

1 Inheritance

1. What would Python display in the land of Oz?

```
class Elphaba(object):
    magic = 'Defying Gravity'

    def __init__(self, magic):
        self.magic = magic

    def spell(self):
        return type(self).magic + ' and ' + self.magic

class Glinda(Elphaba):
    magic = 'Popular'

    def __init__(self, magic):
        Elphaba.__init__(self, 'good ' + magic)

elphaba = Elphaba('Wicked')
```

(a) `>>> Elphaba.magic`

(b) `>>> elphaba.magic`

(c) `>>> elphaba.spell()`

(d) `>>> Elphaba.spell()`

(e) `>>> Elphaba.spell(elphaba)`

Created by Cadence Loh, Ana Abell, Tien-Yu Hsu, and Rajoshi Basu. feedback form!
Provided by CSM 61A contributors under CC BY 4.0/3-clause BSD.

(f)
```
>>> glinda = Glinda('Power')
>>> Glinda.magic
```

(g)
```
>>> glinda.magic
```

(h)
```
>>> glinda.spell()
```

2. What would Python display? Write the result of executing the following code and prompts. If nothing would happen, write "Nothing". If an error occurs, write "Error".

```
class ForceWielder():
    force = 25

    def __init__(self, name):
        self.name = name

    def train(self, other):
        other.force += self.force / 5

    def __str__(self):
        return self.name

class Jedi(ForceWielder):
    lightsaber = "blue"

    def __str__(self):
        return "Jedi " + self.name

    def __repr__(self):
        return f"Jedi({repr(self.name)})"

class Sith(ForceWielder):
    lightsaber = "red"
    num_sith = 0

    def __init__(self, name):
        super().__init__(name)
        Sith.num_sith += 1
        if self.num_sith != 2:
            print("Two there should be. No more, no less.")

    def __str__(self):
        return "Darth " + self.name

    def __repr__(self):
        return f"Sith({repr(self.name)})"
```

```
>>> anakin = Jedi("Anakin")
>>> anakin.lightsaber, anakin.force


>>> obiwan = Jedi("Obi-wan")
>>> anakin.master = obiwan
>>> anakin.master


>>> Jedi.master


>>> obiwan.force += anakin.force
>>> obiwan.force, anakin.force


>>> obiwan.train(anakin)
>>> obiwan.force, anakin.force


>>> Jedi.train(obiwan, anakin)
>>> obiwan.force, anakin.force


>>> sidious = Sith("Sidious")


>>> ForceWielder.train(sidious, anakin)
>>> anakin.lightsaber = "red"
>>> anakin.lightsaber, anakin.force


>>> Jedi.lightsaber


>>> print(Sith("Vader"), Sith("Maul").num_sith)


>>> rey = ForceWielder("Rey")
>>> rey


>>> rey.lightsaber
```

3. Write `TeamBaller`, a subclass of `Baller`. An instance of `TeamBaller` cheers on the team every time it passes a ball/states it doesn't have the ball when it doesn't and returns a boolean of whether or not it passed the ball. If the `TeamBaller` did not pass the ball, it'll say it hasn't done so and return False. Assume `Baller` has a defined *pass_ball* method.

```
class Baller:
    all_players = []
    def __init__(self, name, has_ball = False):
        self.name = name
        self.has_ball = has_ball
        Baller.all_players.append(self)

    def pass_ball(self, other_player):
        if self.has_ball:
            self.has_ball = False
            other_player.has_ball = True
            return True
        else:
            return False

class BallHog(Baller):
    def pass_ball(self, other_player):
        return False

class TeamBaller(__________________):
    """
    >>> alyssa = BallHog('Alyssa')
    >>> cheerballer = TeamBaller('Esther', has_ball=True)
    >>> cheerballer.pass_ball(alyssa)
    Yay!
    True
    >>> cheerballer.pass_ball(alyssa)
    I don't have the ball
    False
    """
    def pass_ball(__________________, ____________________):
        ____________________________________________
        ____________________
            ____________________________________
        __________
            ____________________________________
        ____________________
```

2 Iterators

An **iterable** is any object whose elements can be processed one at a time, such as a list, string, dictionary, or **range**. An **iterator** is an object that keeps track of a position in a sequence and can produce the next element on request.

- **iter**(iterable) returns a *new* iterator over the elements of the iterable. Calling **iter** on an iterator returns that same iterator.
- **next**(iterator) returns the next element and advances the iterator. When no elements remain, it raises StopIteration.

```
>>> s = [3, 4, 5]
>>> t = iter(s)
>>> next(t)
3
>>> next(t)
4
>>> u = iter(s)         # a separate iterator with its own position
>>> next(u)
3
>>> w = t               # not a new iterator: another name for the same one
>>> next(w)
5
>>> next(t)
StopIteration
```

Iterators only move forward, and calling **next** on any name for an iterator advances it for every name. A **for** loop calls **iter** on its iterable and repeatedly calls **next** until StopIteration.

Many built-in functions return iterators that compute their results **lazily**, meaning each element is computed only when **next** asks for it:

- **map**(func, iterable): iterate over func(x) for each x in the iterable
- **filter**(func, iterable): iterate over each x in the iterable for which func(x) is true
- **zip**(first_iter, second_iter): iterate over pairs (x, y)
- **reversed**(sequence): iterate over the sequence in reverse

Functions such as **list**, **sum**, **min**, **max**, **all**, and **any** consume an iterable. **all** stops at the first false value and **any** stops at the first true value, so they may not use up the whole iterator.

4. What is difference between an iterator and an iterable?

5. What Would Python Display? If you think nothing will be displayed, write N/A.

```
>>> a = [1, 2, 3]
>>> x = iter(a)


>>> next(x)


>>> next(x)


>>> y = iter(a)
>>> next(y)


>>> next(x)


>>> next(x)


>>> z = iter(y)
>>> next(z)


>>> [next(y), next(y), next(z)]


>>> a = iter(filter(lambda x: x % 2, map(lambda x: x - 1, range(10))))
>>> next(a)


>>> reduce(lambda x, y: x + y, a)
```

6. WWPD? Write what this python program will print.

```
a = [1, 'A', 'B', 'C', 5, 6, 7, 'D', 'E']
x = iter(a)
for i in range(5 - next(x)):
    next(x)
print(next(x))
y = iter(a)
print(next(y))
z = iter(y)
print(next(z))
```

7. Find an input to the year function that prints the following output: 2 0 2 6

```
def year(a):
    x = iter(a)
    y = iter(a)
    z = iter(x)
    for i in range(next(x)):
        y = iter(a)
        next(y)
    print(next(x))
    print(next(z))
    print(next(y))
    print(next(z))
```

3 Generators

A **generator function** is a function whose body contains **`yield`**. A normal function returns once; a generator function can yield many times. A **generator** is an iterator created automatically by calling a generator function.

```
>>> def plus_minus(x):
...     yield x
...     yield -x
>>> t = plus_minus(3)     # no body code runs yet
>>> t
<generator object plus_minus ...>
>>> next(t)               # run until the first yield
3
>>> next(t)               # resume after the first yield
```

```
-3
>>> next(t)                # the body finished
StopIteration
```

Calling a generator function does not run its body. The body runs only when **next** is called, and it pauses at each **yield**, keeping its local variables and its place in the code until the next call to **next**. When the body finishes (or reaches **return**), StopIteration is raised. Each call to a generator function creates an independent generator.

yield from <iterable> yields each element of the iterable in order, and is shorthand for a loop:

```
def a():                        def b():
    yield from range(3)             for x in range(3):
                                        yield x
```

Recursive generators combine **yield from** (to pass on everything the recursive call produces) with ordinary **yield** (to add values of their own).

8. Given the following code block, what is output by the lines that follow?

```
def foo():
    a = 0
    if a == 0:
        print("Hello")
        yield a
        print("World")
```

```
>>> foo()
```

```
>>> foo_gen = foo()
>>> next(foo_gen)
```

```
>>> next(foo_gen)
```

```
>>> for i in foo():
...   print(i)
```

9. Write a generator function that takes in an iterator `it` and yields the running total of the elements produced by `it`.

```
def accumulate(it):
    """
    >>> def all_ints():
    ...     i = 0
    ...     while True:
    ...         yield i
    ...         i += 1
    >>> a = accumulate(all_ints())
    >>> [next(a) for x in range(6)]
    [0, 1, 3, 6, 10, 15]
    """
```

10. Write a generator that, given m (the amount of money you have), pc (the cost of one pear), and ac (the cost of one apple) yields all possible combinations of fruit that you can buy that uses up the most of your money. In other words, each combination of fruits should not result in enough money left over to buy another fruit. Combinations of the same number of each fruit in different orders is okay. It is also okay if each combination has an extra space at the end.

```
def fruitOptions(m, pc, ac):
    """
    >>> print(list(fruitOptions(10, 2, 5)))
    ['pear pear pear pear pear ', 'pear pear apple ', 'pear apple pear
       ',
     'apple pear pear ', 'apple apple ']
    """
    if ____________________________________:
        yield ""
    if m >= pc:
        for ____________________________________________:
            ____________________________________
    if m >= ac:
        for ____________________________________________:
            ____________________________________________
```